MATH 8090: Regression with Time Series Errors, Unit Root Test, Spurious Correlation and Prewhitening

Whitney Huang, Clemson University

10/14-10/16/2026

Contents

Regression with Time Series Errors . . . 2
Plot the data . . . 2
Fit an OLS an examine the residuals . . . 2
Fit a GLS with an AR(1) error structure . . . 4
Comparison of Two-Step and One-Step Estimation Procedures . . . 6
Simulating Regression Data with ARMA Errors . . . 6
Simulated time series data . . . 6
Step 1: Perform OLS regression . . . 6
Step 2: Fit ARMA model to OLS residuals . . . 7
Step 3: GLS regression using correlation structure from ARMA model . . . 8
One-step MLE . . . 9
Summarize the simulation results . . . 10
Lake Huron Example . . . 15
A two-step fit . . . 15
One-step MLE fit . . . 16
Comparing CIs . . . 16
Unit Root Test Examples . . . 16
Simulating the Two Processes . . . 17
OLS Regression for Unit Root Testing . . . 17
Visualizing the OLS Regressions . . . 19
Spurious Correlation and Prewhitening . . . 20
Spurious Correlations: Inflating Type I error rate . . . 21
Spurious Correlations: Example I . . . 22
Spurious Correlations: An Example with Milk and Electricity Data . . . 24
Detrend and remove seasonality by differencing and applying prewhitening . . . 26
References . . . 28

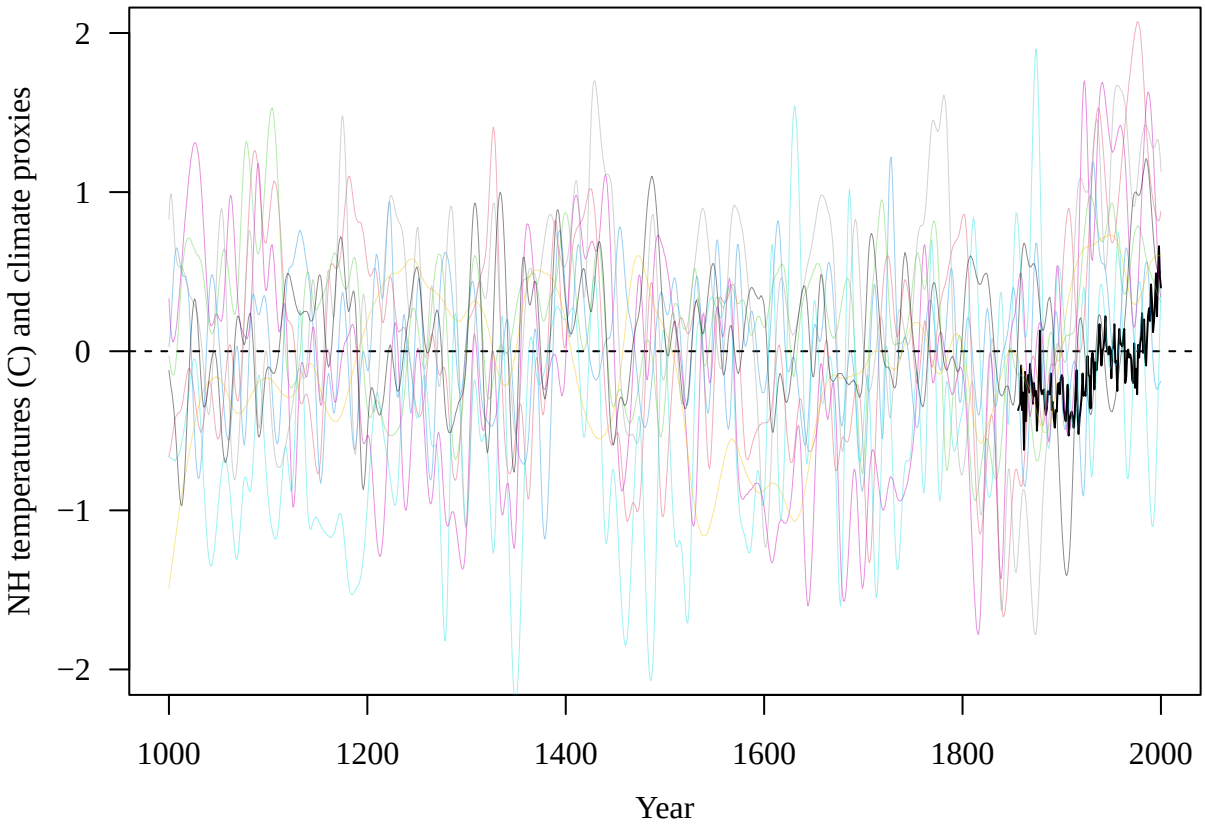
Regression with Time Series Errors

Let us present a brief data analysis of Northern Hemisphere temperatures and tree ring proxies (Jones and Mann (2004)) to illustrate regression with time series errors.

Plot the data

```
library(faraway)
# Load Northern Hemisphere temperature and climate proxy data
data(globwarm)

# Plot temperature anomalies over time
par(las = 1, mgp = c(2.4, 1, 0), mar = c(3.6, 4, 1, 0.6),
    family = "serif")
plot(globwarm$year, globwarm$nhtemp, type = "l", ylim = c(-2, 2),
     xlab = "Year", ylab = "NH temperatures (C) and climate proxies")
abline(h = 0, lty = 2)
# Overlay climate proxy series with semi-transparent lines
library(scales)
for (i in 2:9) lines(globwarm$year, globwarm[, i], col = alpha(i, 0.45), lwd = 0.5)
```



Fit an OLS an examine the residuals

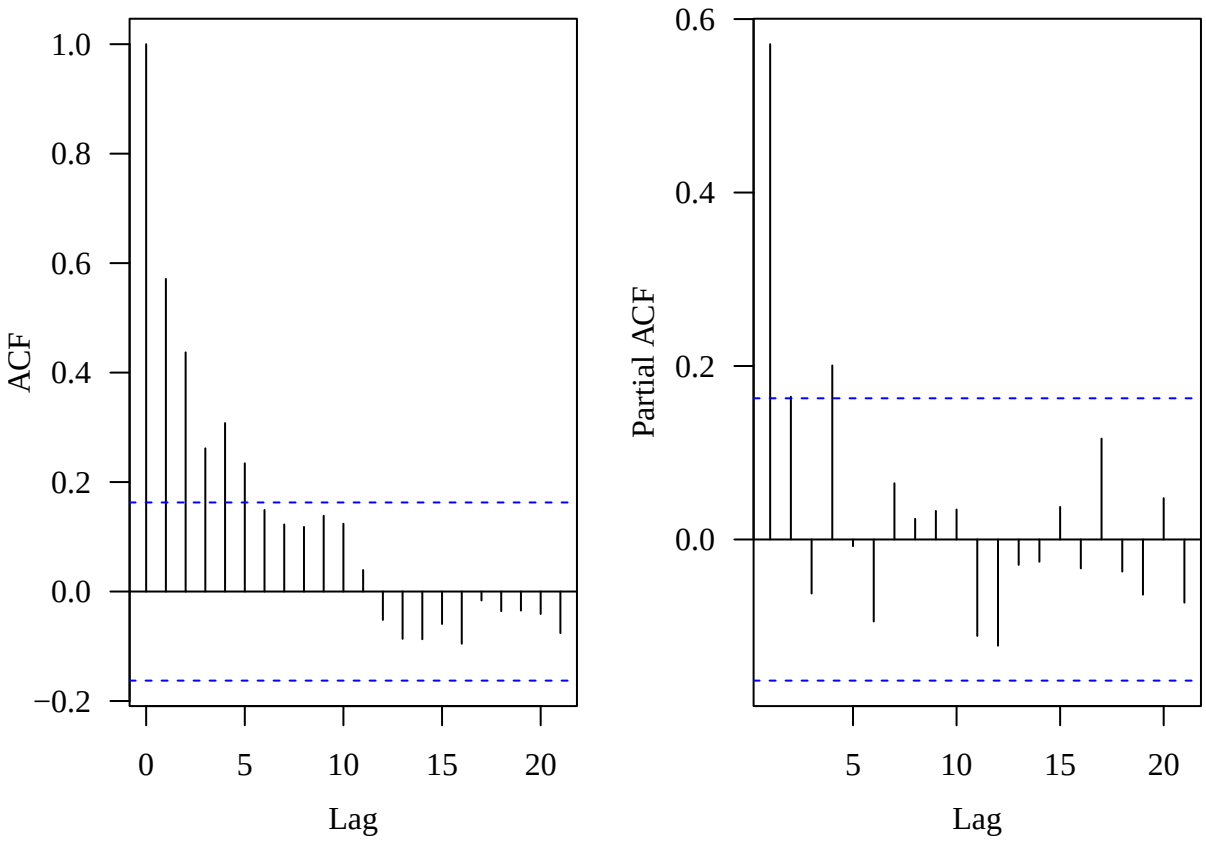
We fit a multiple linear regression model using eight climate proxies to explain Northern Hemisphere temperature variations. The residual ACF and PACF are then examined to assess whether temporal dependence

remains after accounting for these predictors.

```
# Fit a multiple linear regression using eight climate proxies
lmod <- lm(nhtemp ~ wusa + jasper + westgreen + chesapeake
           + tornetrask + urals + mongolia + tasman, data = globwarm)
summary(lmod)
```

```
##
## Call:
## lm(formula = nhtemp ~ wusa + jasper + westgreen + chesapeake +
##     tornetrask + urals + mongolia + tasman, data = globwarm)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.43668 -0.11170  0.00698  0.10176  0.65352
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.242555   0.027011  -8.980 1.97e-15 ***
## wusa         0.077384   0.042927   1.803 0.073647 .
## jasper      -0.228795   0.078107  -2.929 0.003986 **
## westgreen    0.009584   0.041840   0.229 0.819168
## chesapeake  -0.032112   0.034052  -0.943 0.347346
## tornetrask   0.092668   0.045053   2.057 0.041611 *
## urals        0.185369   0.091428   2.027 0.044567 *
## mongolia     0.041973   0.045794   0.917 0.360996
## tasman       0.115453   0.030111   3.834 0.000192 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1758 on 136 degrees of freedom
##   (856 observations deleted due to missingness)
## Multiple R-squared:  0.4764, Adjusted R-squared:  0.4456
## F-statistic: 15.47 on 8 and 136 DF,  p-value: 5.028e-16
```

```
# Examine residual autocorrelation using ACF and PACF
par(las = 1, mgp = c(2.4, 1, 0), mar = c(3.6, 4, 1, 0.6), mfrow = c(1, 2),
    family = "serif")
acf(lmod$residuals)
pacf(lmod$residuals)
```




```
par(mfrow = c(1, 1))
```

Fit a GLS with an AR(1) error structure

The residual ACF from the OLS model suggests temporal dependence. We now use generalized least squares (GLS) with AR(1) errors to account for this correlation while retaining the same climate predictors.

```
library(nlme)

# Fit GLS regression with AR(1) errors
glmod <- gls(nhtemp ~ wusa + jasper + westgreen + chesapeake +
               tornetrask + urals + mongolia + tasman,
             correlation = corAR1(form = ~ year),
             data = na.omit(globwarm))

# Summarize coefficient estimates and model fit
summary(glmod)
```

```
## Generalized least squares fit by REML
##   Model: nhtemp ~ wusa + jasper + westgreen + chesapeake + tornetrask +      urals + mongolia + tasma
##   Data: na.omit(globwarm)
##         AIC       BIC   logLik
##   -108.2074 -76.16822 65.10371
##
## Correlation Structure: AR(1)
```

```
##  Formula: ~year
##  Parameter estimate(s):
##       Phi
## 0.7109922
##
## Coefficients:
##                   Value  Std.Error   t-value p-value
## (Intercept) -0.23010624 0.06702406 -3.433188  0.0008
## wusa         0.06673819 0.09877211  0.675678  0.5004
## jasper      -0.20244335 0.18802773 -1.076668  0.2835
## westgreen   -0.00440299 0.08985321 -0.049002  0.9610
## chesapeake  -0.00735289 0.07349791 -0.100042  0.9205
## tornetrask   0.03835169 0.09482515  0.404446  0.6865
## urals        0.24142199 0.22871028  1.055580  0.2930
## mongolia     0.05694978 0.10489786  0.542907  0.5881
## tasman       0.12034918 0.07456983  1.613913  0.1089
##
##  Correlation:
##             (Intr) wusa   jasper wstgrn chespk trntrs urals  mongol
## wusa        -0.517
## jasper      -0.058 -0.299
## westgreen    0.330 -0.533  0.121
## chesapeake   0.090 -0.314  0.230  0.147
## tornetrask -0.430  0.499 -0.197 -0.328 -0.441
## urals       -0.110 -0.142 -0.265  0.075 -0.064 -0.346
## mongolia     0.459 -0.437 -0.205  0.217  0.449 -0.343 -0.371
## tasman       0.037 -0.322  0.065  0.134  0.116 -0.434  0.416 -0.017
##
## Standardized residuals:
##         Min          Q1         Med          Q3         Max
## -2.31122523 -0.53484054  0.02342908  0.50015642  2.97224724
##
## Residual standard error: 0.204572
## Degrees of freedom: 145 total; 136 residual
```

```r
# Confidence intervals for variance and AR(1) parameters
intervals(glmod, which = "var-cov")
```

```
## Approximate 95% confidence intervals
##
##  Correlation structure:
##         lower      est.     upper
## Phi 0.5099757 0.7109922 0.8383747
##
##  Residual standard error:
##     lower      est.     upper
## 0.1540712 0.2045720 0.2716258
```

Interpretation: Unlike OLS, GLS explicitly models serial correlation in the errors. The estimated AR(1) parameter describes the strength of dependence between consecutive years, while the confidence interval helps assess its uncertainty.

Comparison of Two-Step and One-Step Estimation Procedures

Simulating Regression Data with ARMA Errors

We simulate $N = 500$ datasets from a linear regression model with ARMA(1,1) errors. The regression coefficients are fixed at $\beta_0 = 3$ and $\beta_1 = 0.5$, allowing us to evaluate how different estimation methods perform under serially correlated errors.

$$y_t = 3 + 0.5x_t + \eta_t,$$

$\eta_t = 0.8\eta_{t-1} + Z_t - 0.4Z_{t-1}$, $Z_t \sim \mathrm{N}(0, 1)$.

Simulated time series data

```
set.seed(1234)

# Set simulation parameters
N <- 500
n <- 200
x <- rnorm(n, 10, 2)

# Specify the true regression relationship
true_beta <- c(3, 0.5)
mean <- true_beta[1] + true_beta[2] * x

# Generate N independent ARMA(1,1) error series
err <- replicate(N, arima.sim(n = n,
                 model = list(ar = 0.8, ma = -0.4), sd = 1))

# Construct regression responses with correlated errors
y <- mean + err
```

Step 1: Perform OLS regression

We first fit OLS regression to each of the 500 simulated datasets, ignoring the ARMA(1,1) error structure. We then examine the residual ACF and PACF from the first dataset to assess the remaining temporal dependence.

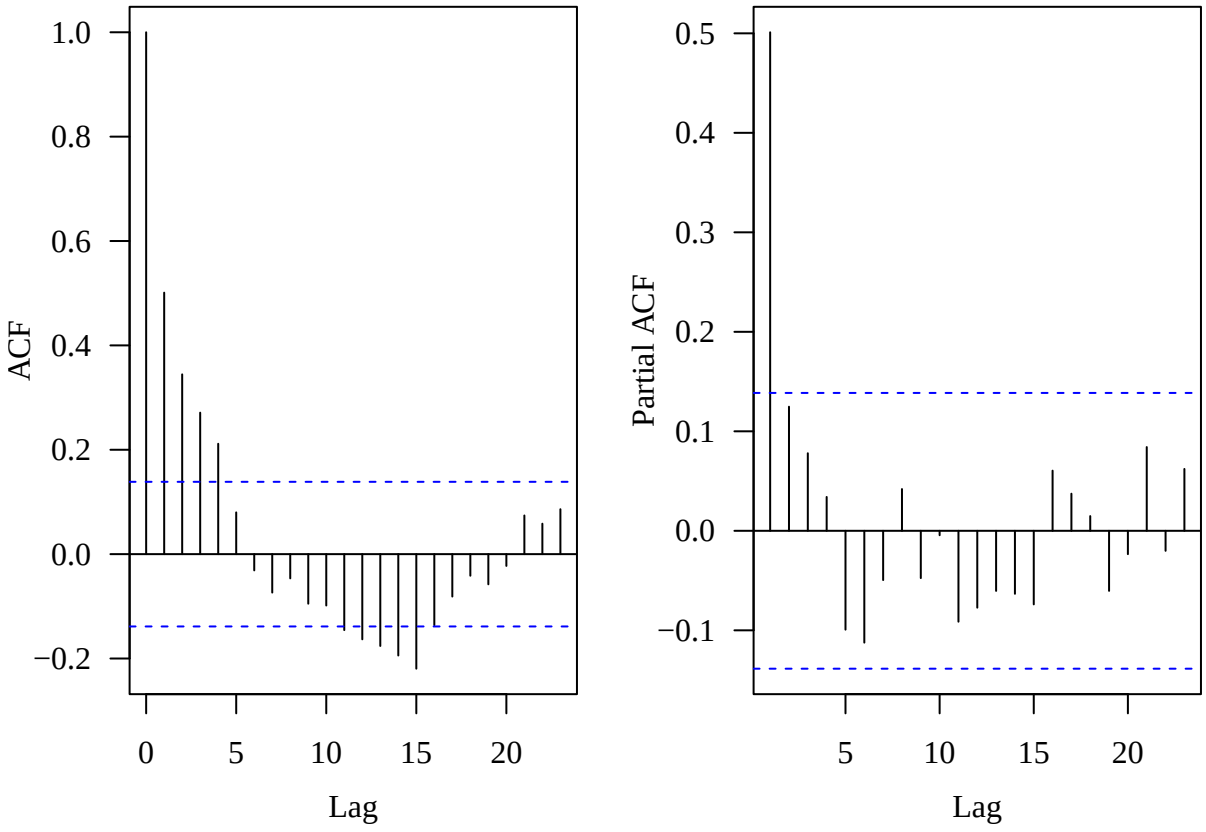
```
# Fit OLS regression to each simulated dataset
ols_fit <- apply(y, 2, function(z) lm(z ~ x))
summary(ols_fit[[1]])
```

```
##
## Call:
## lm(formula = z ~ x)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -3.5722 -0.7243  0.0514  0.8700  2.9476
##
## Coefficients:
```

```
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  3.33404    0.40767   8.178 3.41e-14 ***
## x            0.46400    0.04039  11.487  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.163 on 198 degrees of freedom
## Multiple R-squared:  0.3999, Adjusted R-squared:  0.3969
## F-statistic: 131.9 on 1 and 198 DF,  p-value: < 2.2e-16
```

```r
# Extract residuals from all fitted models
res_ols <- lapply(ols_fit, residuals)

# Examine residual ACF and PACF for the first dataset
par(las = 1, mgp = c(2.4, 1, 0), mar = c(3.6, 4, 1, 0.6),
    mfrow = c(1, 2), family = "serif")
acf(res_ols[[1]])
pacf(res_ols[[1]])
```




```r
par(mfrow = c(1, 1))
```

Step 2: Fit ARMA model to OLS residuals

We now fit an ARMA(1,1) model to the OLS residuals from each simulated dataset. This is the second step of the two-step procedure, allowing us to estimate the temporal dependence parameters ϕ and θ.

```r
library(forecast) # For ARMA model fitting
```

```
## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo
```

```
##
## Attaching package: 'forecast'
```

```
## The following object is masked from 'package:nlme':
##
##     getResponse
```

```r
# Fit ARMA(1,1) models to OLS residuals
arma_fit <- lapply(res_ols, arima,
                   order = c(1, 0, 1), include.mean = FALSE)

# Extract estimated AR and MA coefficients
phi <- sapply(arma_fit, function(x) x$coef["ar1"])
theta <- sapply(arma_fit, function(x) x$coef["ma1"])
```

Interpretation: We can compare the estimated AR and MA coefficients across the 500 simulations with their true values, $\phi = 0.8$ and $\theta = -0.4$, to assess the accuracy and variability of the two-step procedure.

Step 3: GLS regression using correlation structure from ARMA model

Using the ARMA(1,1) parameters estimated from the OLS residuals, we refit the regression model using GLS. The ARMA parameters are held fixed, allowing GLS to account for serial correlation without jointly re-estimating the error structure.

```r
# Fit GLS regression with ARMA(1,1) errors
gls_fit <- list()

for (i in 1:N) {
  # Fix ARMA(1,1) parameters estimated from OLS residuals
  cor_struct <- corARMA(value = c(phi[i], theta[i]),
                        p = 1, q = 1, form = ~ 1,
                        fixed = TRUE)

  # Fit GLS using the fixed correlation structure
  y_each <- y[, i]
  gls_fit[[i]] <- gls(y_each ~ x,
                      correlation = cor_struct,
                      method = "ML")
}

# Examine results for the first dataset
summary(gls_fit[[1]])
```

```
## Generalized least squares fit by maximum likelihood
```

```
##   Model: y_each ~ x
##   Data: NULL
##        AIC      BIC    logLik
##   565.4294 575.3244 -279.7147
##
## Correlation Structure: ARMA(1,1)
##  Formula: ~1
##  Parameter estimate(s):
##       Phi1     Theta1
##  0.7056013 -0.2792677
##
## Coefficients:
##                Value Std.Error   t-value p-value
## (Intercept) 2.645437 0.3624032  7.299707       0
## x           0.534753 0.0325934 16.406810       0
##
##  Correlation:
##   (Intr)
## x -0.885
##
## Standardized residuals:
##          Min           Q1          Med           Q3          Max
## -3.261626892 -0.578995450  0.004997083  0.718677576  2.683243600
##
## Residual standard error: 1.142567
## Degrees of freedom: 200 total; 198 residual
```

```
intervals(gls_fit[[1]])
```

```
## Approximate 95% confidence intervals
##
##  Coefficients:
##                 lower     est.     upper
## (Intercept) 1.9307717 2.645437 3.3601026
## x           0.4704783 0.534753 0.5990276
##
##  Residual standard error:
##    lower     est.    upper
## 1.040725 1.142567 1.266676
```

Interpretation: This two-step procedure accounts for serial correlation when estimating the regression coefficients. Unlike joint ML estimation, the ARMA parameters are not updated during GLS fitting.

One-step MLE

We now fit a regression model with ARMA(1,1) errors using a one-step approach. Unlike two-step feasible GLS, this method jointly estimates the regression coefficients and ARMA parameters by maximum likelihood.

```
# Fit regression with ARMA(1,1) errors using joint ML
mle <- apply(y, 2, arima, order = c(1, 0, 1), xreg = x)

# Examine confidence intervals for the first dataset
confint(mle[[1]])
```

```
##                2.5 %      97.5 %
## ar1        0.5505461  0.89103901
## ma1       -0.5088234 -0.03577565
## intercept  1.9116985  3.34531698
## x          0.4728902  0.60025025
```

Interpretation: Joint ML estimates the regression coefficients and error dependence parameters simultaneously, allowing us to compare its estimation accuracy and uncertainty with OLS and two-step feasible GLS.

Summarize the simulation results

Comparing Regression Coefficient Estimates

We first extract the regression coefficient estimates from OLS, two-step feasible GLS, and joint ML across the 500 simulated datasets.

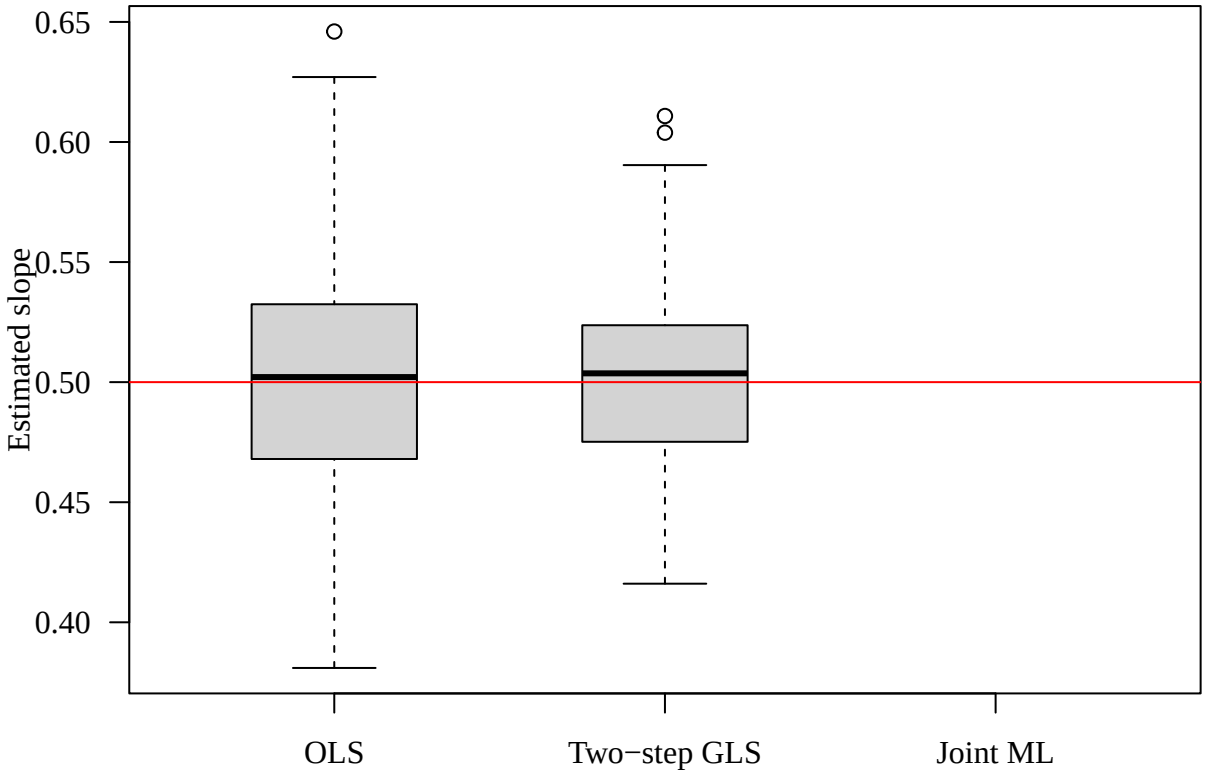
```
# Extract regression coefficients from each method
beta_ols <- t(sapply(ols_fit, coef))
beta_gls <- t(sapply(gls_fit, coef))
beta_mle <- t(sapply(mle, function(x) x$coef[c("intercept", "xreg")]))
```

Comparing the Slope Estimates

We compare the sampling distributions of the estimated slope $\hat{\beta}_1$. The red horizontal line indicates the true value, $\beta_1 = 0.5$.

```
# Compare slope estimates across methods
par(las = 1, mgp = c(2.4, 1, 0), mar = c(3.6, 4, 1, 0.6),
    family = "serif")

boxplot(beta_ols[, 2], beta_gls[, 2], beta_mle[, 2],
        names = c("OLS", "Two-step GLS", "Joint ML"),
        boxwex = 0.5, ylab = "Estimated slope")
abline(h = 0.5, col = "red")
```



We summarize the bias and standard deviation of the slope estimates to assess their accuracy and efficiency.

```
# Compute empirical bias and standard deviation
bias <- c(mean(beta_ols[, 2]), mean(beta_gls[, 2]),
          mean(beta_mle[, 2])) - 0.5

sd <- c(sd(beta_ols[, 2]), sd(beta_gls[, 2]),
        sd(beta_mle[, 2]))

rbind(Bias = bias, SD = sd)
```

```
##               [,1]         [,2] [,3]
## Bias -0.0004185194 0.0008856242   NA
## SD    0.0464772791 0.0346610824   NA
```

Confidence Interval Coverage for the Slope

Next, we evaluate how often the nominal 95% confidence intervals contain the true slope, $\beta_1 = 0.5$, and compare their average widths.

```
# Extract 95% confidence intervals for the slope
CI_beta_ols <- t(sapply(ols_fit, function(x) confint(x)["x", ]))
CI_beta_gls <- t(sapply(gls_fit, function(x) confint(x)["x", ]))
CI_beta_mle <- t(sapply(mle, function(x) confint(x)["x", ]))
```

```
# Compute empirical coverage probabilities
coverage <- c(
  OLS = mean(CI_beta_ols[, 1] <= 0.5 & CI_beta_ols[, 2] >= 0.5),
  GLS = mean(CI_beta_gls[, 1] <= 0.5 & CI_beta_gls[, 2] >= 0.5),
```

```
  MLE = mean(CI_beta_mle[, 1] <= 0.5 & CI_beta_mle[, 2] >= 0.5)
)

# Compute average confidence interval widths
width <- c(
  OLS = mean(CI_beta_ols[, 2] - CI_beta_ols[, 1]),
  GLS = mean(CI_beta_gls[, 2] - CI_beta_gls[, 1]),
  MLE = mean(CI_beta_mle[, 2] - CI_beta_mle[, 1])
)

rbind(Coverage = coverage, Avg_Width = width)
```

```
##                 OLS       GLS       MLE
## Coverage  0.9080000 0.9360000 0.9360000
## Avg_Width 0.1615202 0.1290678 0.1290107
```

Interpretation: Ideally, the empirical coverage should be close to 95%. Ignoring serial correlation may lead to misleading OLS confidence intervals, while two-step GLS and joint ML account for the ARMA error structure. The two-step GLS intervals, however, do not fully account for uncertainty in the estimated correlation parameters.

Comparing AR(1) Parameter Estimates

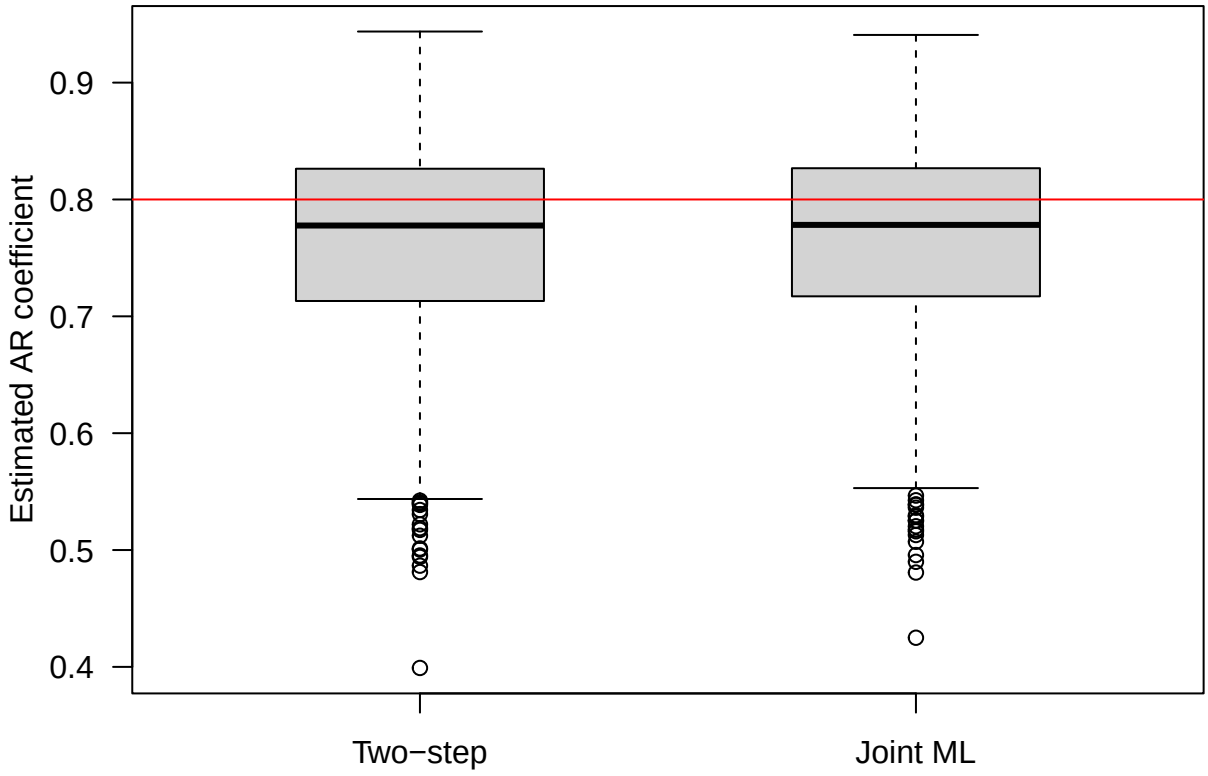
We now compare estimates of the ARMA(1,1) dependence parameters from the two-step procedure and joint ML.

```
# Extract ARMA parameter estimates
arma_2step <- cbind(phi, theta)
arma_1step <- t(sapply(mle, function(x) x$coef[c("ar1", "ma1")]))
```

We first examine the AR coefficient ϕ, whose true value is 0.8.

```
# Compare AR(1) coefficient estimates
par(las = 1, mgp = c(2.4, 1, 0), mar = c(3.6, 4, 1, 0.6))

boxplot(arma_2step[, 1], arma_1step[, 1],
        names = c("Two-step", "Joint ML"),
        boxwex = 0.5, ylab = "Estimated AR coefficient")
abline(h = 0.8, col = "red")
```




```
# Compute empirical bias and standard deviation
bias_phi <- colMeans(arma_2step[, 1, drop = FALSE])
bias_phi <- c(mean(arma_2step[, 1]),
              mean(arma_1step[, 1])) - 0.8

sd_phi <- c(sd(arma_2step[, 1]),
            sd(arma_1step[, 1]))

rbind(Bias = bias_phi, SD = sd_phi)
```

```
##             [,1]        [,2]
## Bias -0.03764122 -0.03597188
## SD    0.08958435  0.08890410
```

Confidence Interval Coverage for the AR Coefficient

We examine the empirical coverage and average width of the 95% confidence intervals for $\phi = 0.8$.

```
# Extract confidence intervals for the AR coefficient
CI_phi_mle <- t(sapply(mle, function(x) confint(x)["ar1", ]))
CI_phi_2step <- t(sapply(arma_fit, function(x) confint(x)["ar1", ]))
```

```
# Compare coverage and average interval width
coverage_phi <- c(
  Two_step = mean(CI_phi_2step[, 1] <= 0.8 &
                  CI_phi_2step[, 2] >= 0.8),
  Joint_ML = mean(CI_phi_mle[, 1] <= 0.8 &
                  CI_phi_mle[, 2] >= 0.8)
```

```
)

width_phi <- c(
  Two_step = mean(CI_phi_2step[, 2] - CI_phi_2step[, 1]),
  Joint_ML = mean(CI_phi_mle[, 2] - CI_phi_mle[, 1])
)

rbind(Coverage = coverage_phi, Avg_Width = width_phi)
```

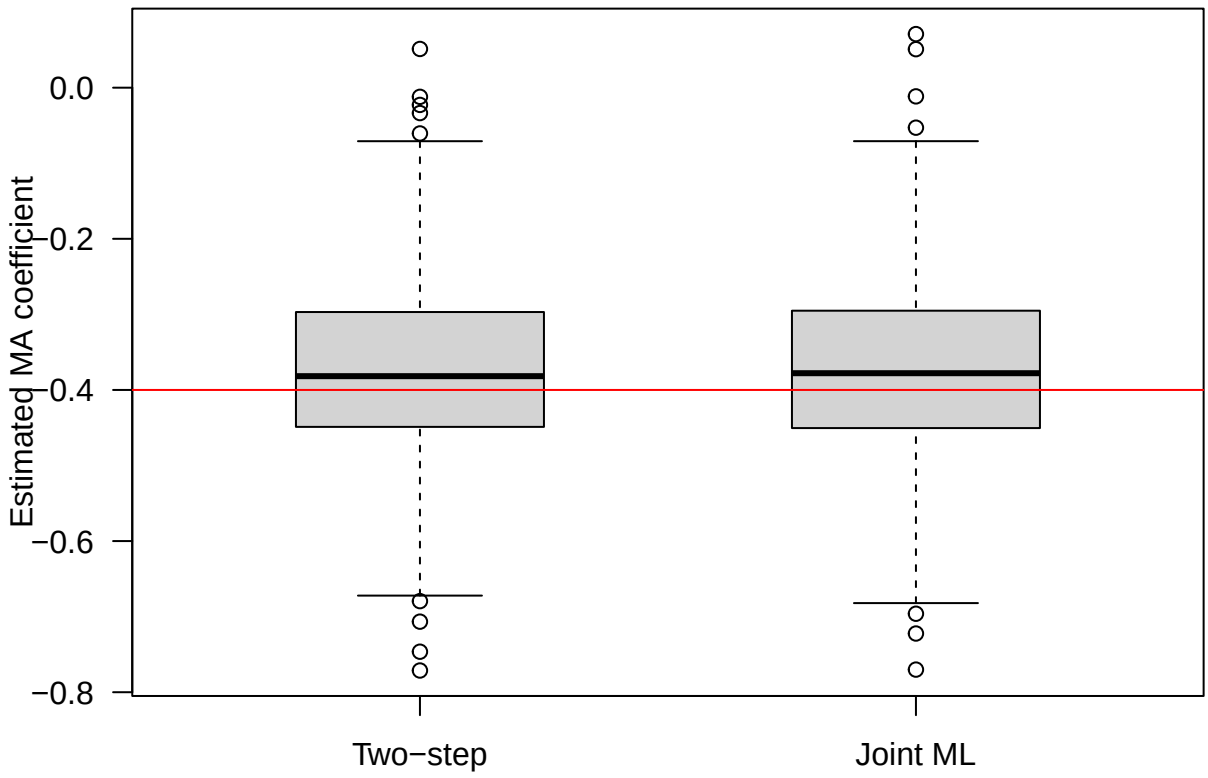
```
##            Two_step  Joint_ML
## Coverage  0.9660000 0.9620000
## Avg_Width 0.3304249 0.3275391
```

Comparing MA(1) Parameter Estimates

Finally, we compare estimates of the MA coefficient θ, whose true value is -0.4.

```
# Compare MA(1) coefficient estimates
par(las = 1, mgp = c(2.4, 1, 0), mar = c(3.6, 4, 1, 0.6))

boxplot(arma_2step[, 2], arma_1step[, 2],
        names = c("Two-step", "Joint ML"),
        boxwex = 0.5, ylab = "Estimated MA coefficient")
abline(h = -0.4, col = "red")
```




```
# Compute empirical bias and standard deviation
bias_theta <- c(mean(arma_2step[, 2]),
                mean(arma_1step[, 2])) - (-0.4)
```

```
sd_theta <- c(sd(arma_2step[, 2]),
              sd(arma_1step[, 2]))

rbind(Bias = bias_theta, SD = sd_theta)
```

```
##            [,1]       [,2]
## Bias 0.02843072 0.03109514
## SD   0.12236128 0.12224490
```

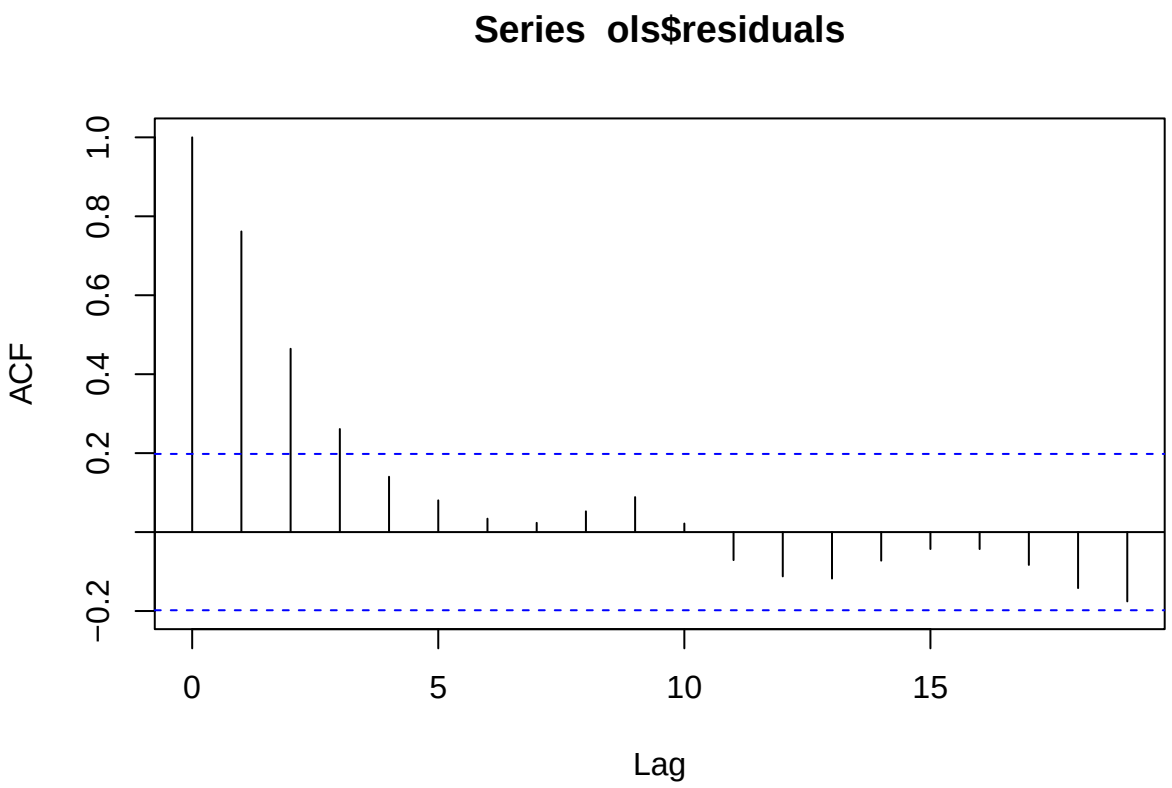
Lake Huron Example

A two-step fit

```
data(LakeHuron)
years <- time(LakeHuron)
ols<- lm(LakeHuron ~ years)
ols$coefficients
```

```
##  (Intercept)        years
## 625.55491791  -0.02420111
```

```
acf(ols$residuals)
```



```r
(arma1 <- arima(ols$residuals, order = c(2, 0, 0), include.mean = FALSE))
```

```
##
## Call:
## arima(x = ols$residuals, order = c(2, 0, 0), include.mean = FALSE)
##
## Coefficients:
##          ar1      ar2
##       1.0050  -0.2925
## s.e.  0.0976   0.1002
##
## sigma^2 estimated as 0.4572:  log likelihood = -101.26,  aic = 208.51
```

One-step MLE fit

```r
yr <- as.numeric(years)
mle <- arima(LakeHuron, order = c(2, 0, 0), xreg = yr, include.mean = T)
```

Comparing CIs

```r
confint(ols)
```

```
##                    2.5 %      97.5 %
## (Intercept) 610.14291793 640.9669179
## years        -0.03221272  -0.0161895
```

```r
confint(arma1)
```

```
##          2.5 %      97.5 %
## ar1  0.8137180  1.19630830
## ar2 -0.4888881 -0.09606208
```

```r
confint(mle)
```

```
##                   2.5 %        97.5 %
## ar1          0.81348340   1.196124084
## ar2         -0.48806617  -0.094573470
## intercept 589.98096094 651.042029064
## yr          -0.03744267  -0.005694985
```

Unit Root Test Examples

We compare a random walk, which contains a unit root, with white noise, which is stationary.

Simulating the Two Processes

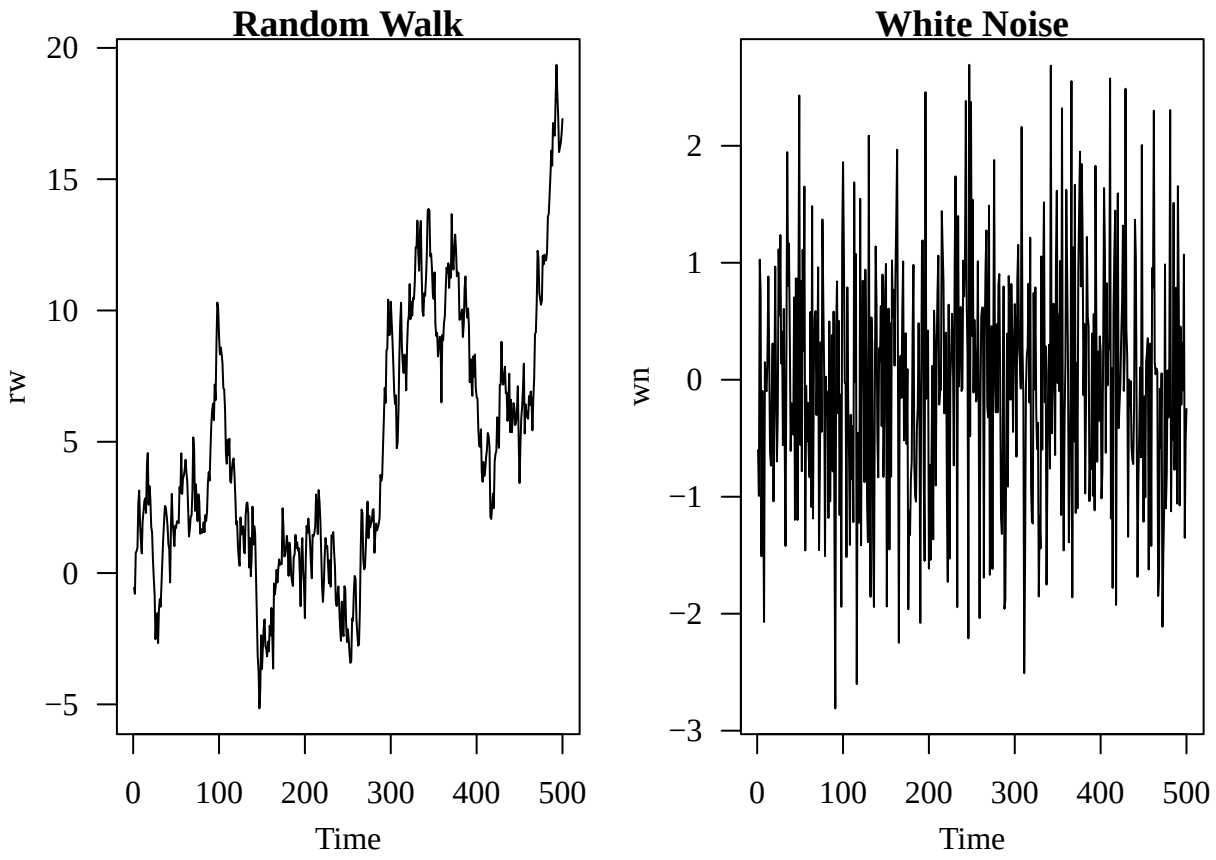
We generate 500 observations from each process and examine their time series plots.

```r
set.seed(123)

# Simulate a random walk and white noise
rw <- cumsum(rnorm(500))
wn <- rnorm(500)

# Compare the two time series
par(las = 1, mgp = c(2.2, 1, 0),
    mar = c(3.6, 3.6, 0.8, 0.6), mfrow = c(1, 2), family = "serif")

ts.plot(rw, main = "Random Walk")
ts.plot(wn, main = "White Noise")
```



OLS Regression for Unit Root Testing

Consider the regression

$$\Delta X_t = \alpha + \gamma X_{t-1} + \varepsilon_t,$$

where $\gamma = \phi - 1$. Testing for a unit root corresponds to

$$H_0 : \gamma = 0 \qquad \text{vs.} \qquad H_1 : \gamma < 0.$$

We fit this regression separately to the two simulated series.

```
# Fit the Dickey-Fuller regression for the random walk
diff.rw <- diff(rw)
n <- length(rw)

ols.rw <- lm(diff.rw ~ rw[1:(n-1)])
summary(ols.rw)
```

```
##
## Call:
## lm(formula = diff.rw ~ rw[1:(n - 1)])
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.67541 -0.62862 -0.01118  0.63805  3.08747
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)    0.10125    0.05973   1.695   0.0906 .
## rw[1:(n - 1)] -0.01438    0.00899  -1.600   0.1102
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9719 on 497 degrees of freedom
## Multiple R-squared:  0.005124,   Adjusted R-squared:  0.003123
## F-statistic:  2.56 on 1 and 497 DF,  p-value: 0.1102
```

```
# Fit the Dickey-Fuller regression for white noise
diff.wn <- diff(wn)

ols.wn <- lm(diff.wn ~ wn[1:(n-1)])
summary(ols.wn)
```

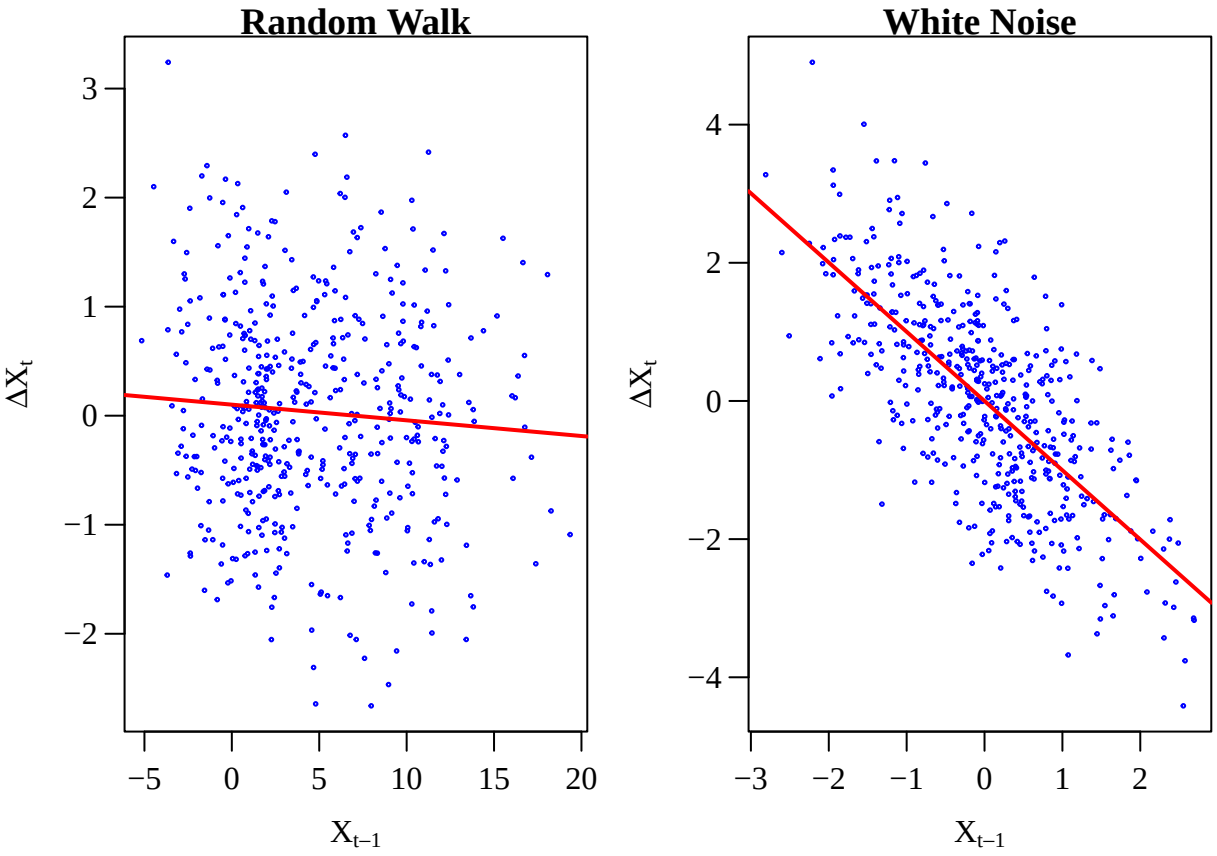
```
##
## Call:
## lm(formula = diff.wn ~ wn[1:(n - 1)])
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.81182 -0.69065  0.00075  0.64461  2.68750
##
## Coefficients:
##                Estimate Std. Error t value Pr(>|t|)
## (Intercept)   -0.001138   0.045329  -0.025     0.98
## wn[1:(n - 1)] -1.002420   0.044843 -22.354   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.013 on 497 degrees of freedom
## Multiple R-squared:  0.5014, Adjusted R-squared:  0.5004
## F-statistic: 499.7 on 1 and 497 DF,  p-value: < 2.2e-16
```

Visualizing the OLS Regressions

```r
# Plot the fitted Dickey-Fuller regressions
par(las = 1, mgp = c(2.2, 0.7, 0),
    mar = c(3.6, 3.6, 0.8, 0.6), mfrow = c(1, 2), family = "serif")

plot(rw[-n], diff.rw,
     xlab = expression(X[t-1]), ylab = expression(Delta*X[t]),
     main = "Random Walk", cex = 0.25, col = "blue")
abline(ols.rw, col = "red", lwd = 2)

plot(wn[-n], diff.wn,
     xlab = expression(X[t-1]), ylab = expression(Delta*X[t]),
     main = "White Noise", cex = 0.25, col = "blue")
abline(ols.wn, col = "red", lwd = 2)
```



Augmented Dickey–Fuller (ADF) Test

The ADF test extends the Dickey–Fuller regression by including lagged differences to account for serial correlation:

$$\Delta X_t = \alpha + \gamma X_{t-1} + \sum_{j=1}^{p} \delta_j \Delta X_{t-j} + \varepsilon_t.$$

We apply the ADF test to both simulated series.

```r
library(tseries)

# ADF test for the random walk
adf.test(rw)
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  rw
## Dickey-Fuller = -1.9203, Lag order = 7, p-value = 0.612
## alternative hypothesis: stationary
```

```r
# ADF test for white noise
adf.test(wn)
```

```
## Warning in adf.test(wn): p-value smaller than printed p-value
```

```
##
##  Augmented Dickey-Fuller Test
##
## data:  wn
## Dickey-Fuller = -7.8953, Lag order = 7, p-value = 0.01
## alternative hypothesis: stationary
```

The OLS regression helps illustrate the behavior of the slope coefficient, but valid unit-root inference requires Dickey–Fuller critical values, rather than ordinary OLS t-tests.

Spurious Correlation and Prewhitening

Consider two time series related by

$$Y_t = X_{t-2} + \varepsilon_t,$$

where $X_t \overset{i.i.d.}{\sim} N(0,1)$, $\varepsilon_t \overset{i.i.d.}{\sim} N(0, 0.25)$, and the two sequences are independent. We simulate the series and examine their cross-correlation function (CCF) to identify the two-period delay.

```r
library(TSA)
```

```
## Registered S3 methods overwritten by 'TSA':
##   method       from
##   fitted.Arima forecast
##   plot.Arima   forecast
```

```
##
## Attaching package: 'TSA'
```

```
## The following objects are masked from 'package:stats':
##
##     acf, arima
```

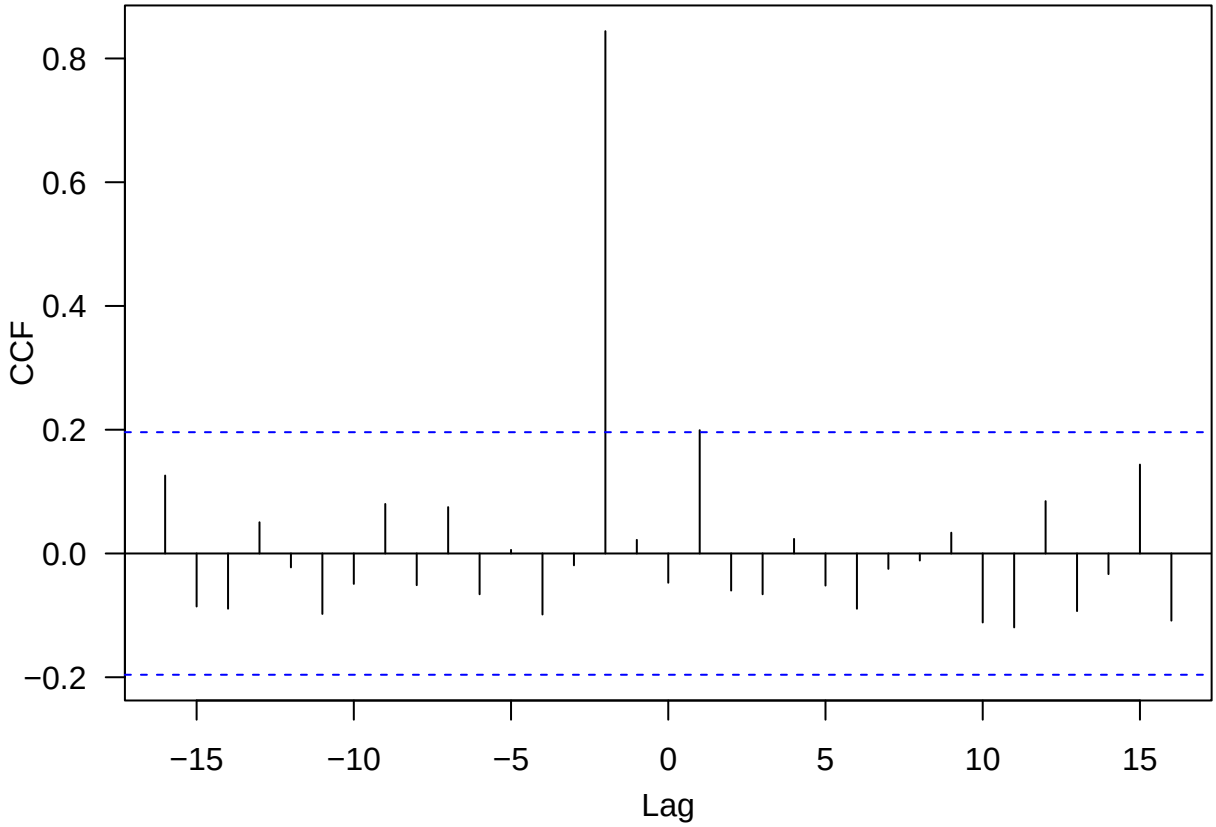
```
## The following object is masked from 'package:utils':
##
##     tar
```

```r
set.seed(123)

# Simulate two series with a two-period lagged relationship
n <- 105
X <- rnorm(n)
Y <- zlag(X, 2) + 0.5 * rnorm(n)

# Remove initial observations and align the series
X <- ts(X[-(1:5)], start = 1, frequency = 1)
Y <- ts(Y[-(1:5)], start = 1, frequency = 1)

# Examine the cross-correlation function
par(las = 1, mgp = c(2.2, 1, 0), mar = c(3.6, 3.6, 0.8, 0.6))
ccf(X, Y, ylab = "CCF")
```



Interpretation: The CCF should exhibit a prominent peak at lag +2 under R's convention, indicating that X_t leads Y_t by two periods.

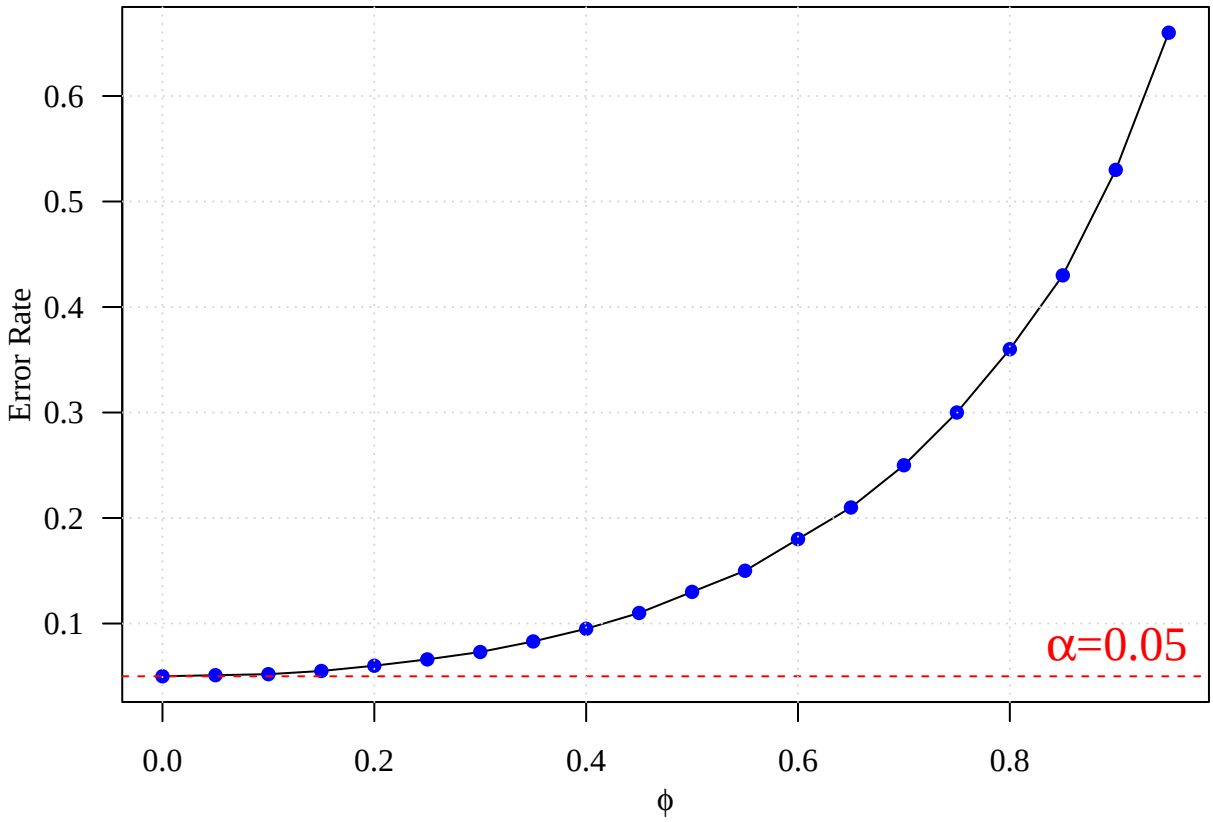
Spurious Correlations: Inflating Type I error rate

Even when two time series are independent, strong autocorrelation within each series can produce misleadingly large sample cross-correlations. We illustrate how the actual Type I error rate increases with the AR(1) coefficient ϕ, when the usual independence-based correlation test is applied to two independent AR(1) series having the same ϕ.

```r
# Compute the approximate Type I error rate for different AR(1) coefficients
phi <- seq(0, 0.95, 0.05)
rejection <- 2 * (1 - pnorm(qnorm(0.975) *
               sqrt((1 - phi^2) / (1 + phi^2))))

M <- signif(rbind(phi, rejection), 2)
rownames(M) <- c("phi", "Error Rate")

# Plot the error rate against the AR(1) coefficient
par(las = 1, mgp = c(2.2, 1, 0), mar = c(3.6, 3.6, 0.8, 0.6),
    family = "serif")
plot(M[1, ], M[2, ], type = "l",
     xlab = expression(phi), ylab = "Error Rate")
points(M[1, ], M[2, ], pch = 16, col = "blue")
abline(h = 0.05, lty = 2, col = "red")
legend("bottomright", legend = expression(paste(alpha, "=", 0.05)),
       bty = "n", text.col = "red", cex = 1.5)
grid()
```



Interpretation: As ϕ increases, the actual Type I error rate can greatly exceed the nominal 5% level. This illustrates why conventional correlation tests can be misleading when serial dependence is ignored.

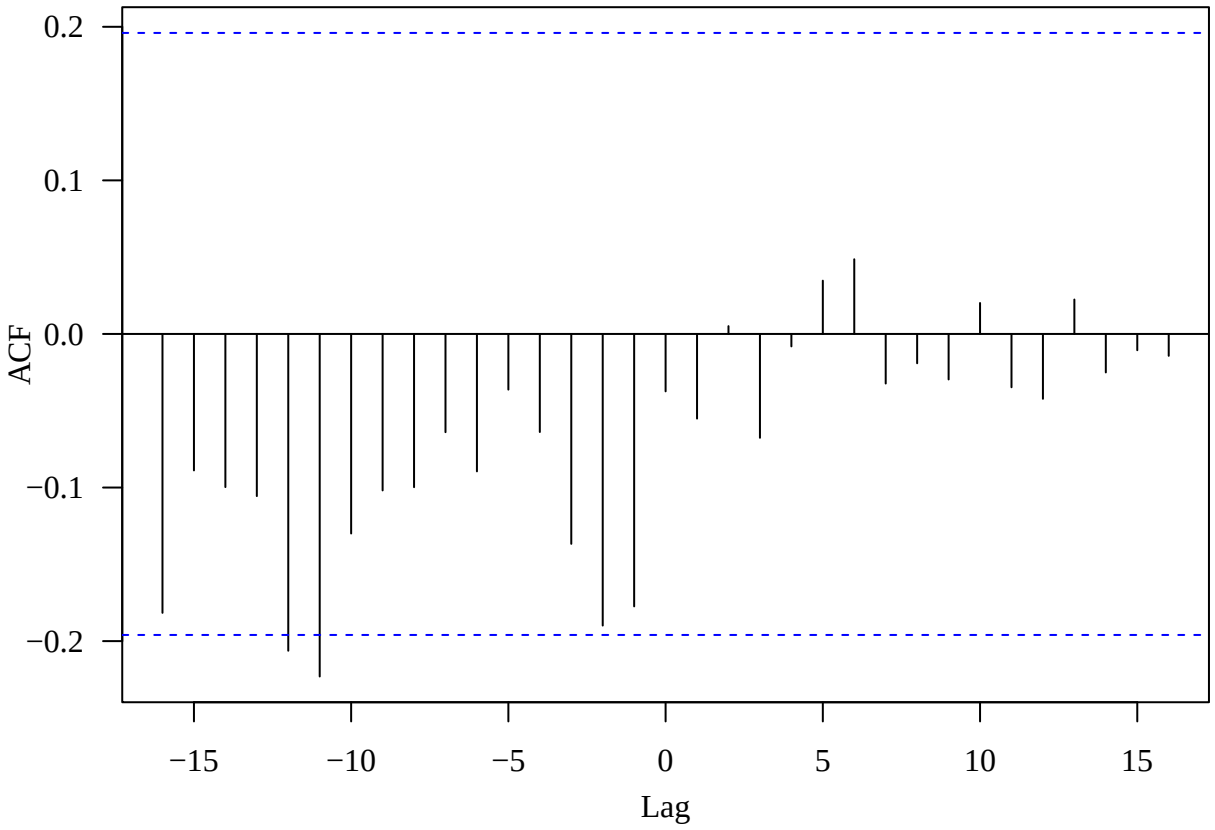
Spurious Correlations: Example I

We generate two independent AR(1) processes with different levels of serial dependence. We then compare their raw CCF with the prewhitened CCF.

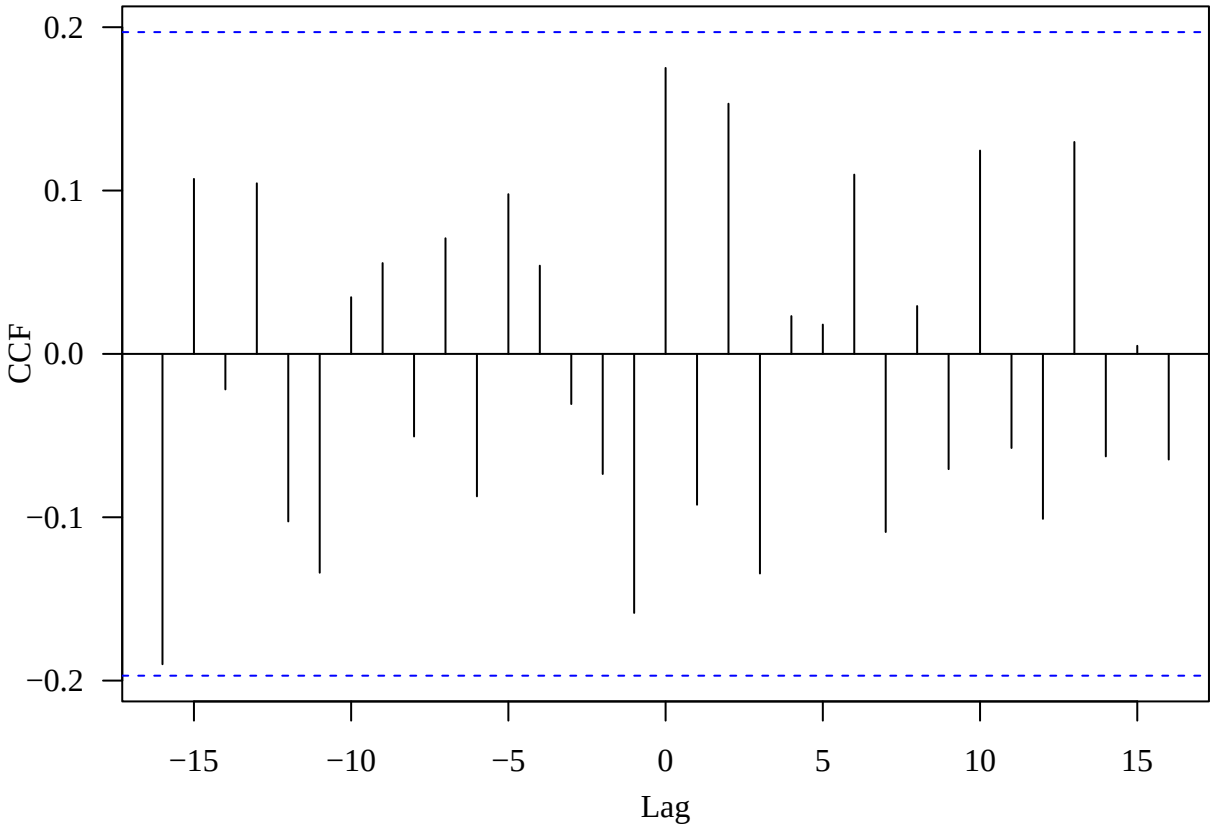
```r
# Simulate two independent AR(1) series
x <- arima.sim(n = 100, list(ar = 0.9))
y <- arima.sim(n = 100, list(ar = 0.01))

# Compare raw and prewhitened cross-correlations
par(las = 1, mgp = c(2.2, 1, 0), mar = c(3.6, 3.6, 0.8, 0.6),
    mfrow = c(1, 1), family = "serif")

ccf(x, y)
```




```r
prewhiten(x, y)
```



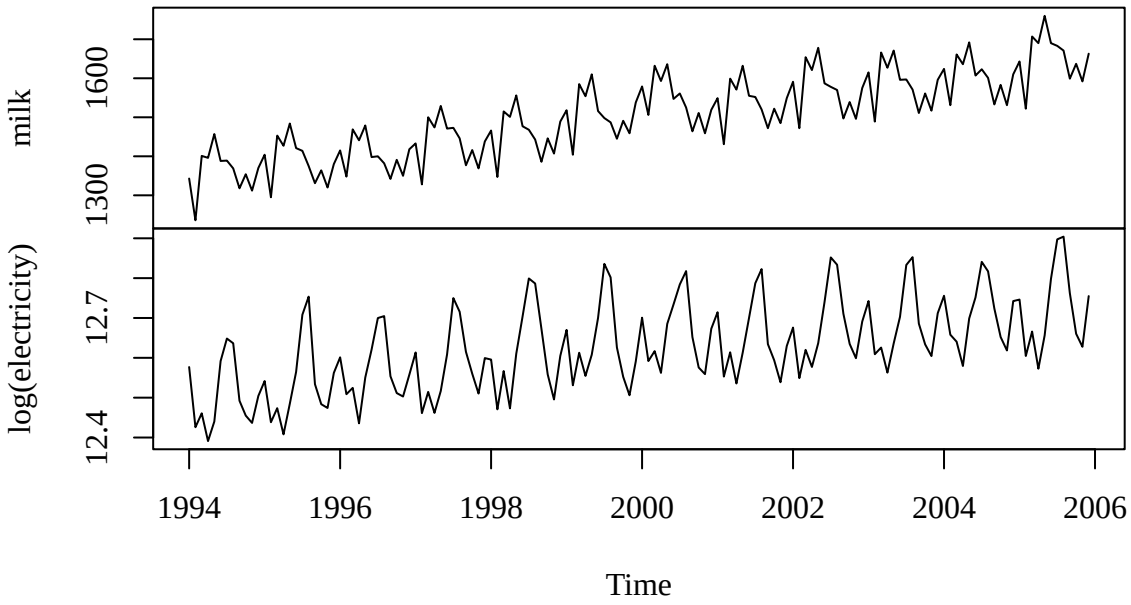
Interpretation: Although the two series are independent, the raw CCF may exhibit seemingly significant correlations because of serial dependence. Prewhitening removes the autocorrelation structure of the input series, helping reveal whether genuine cross-series dependence remains.

Spurious Correlations: An Example with Milk and Electricity Data

We now examine monthly milk production and electricity consumption. Both series may exhibit trend and seasonality, potentially producing strong cross-correlations even without a meaningful direct relationship.

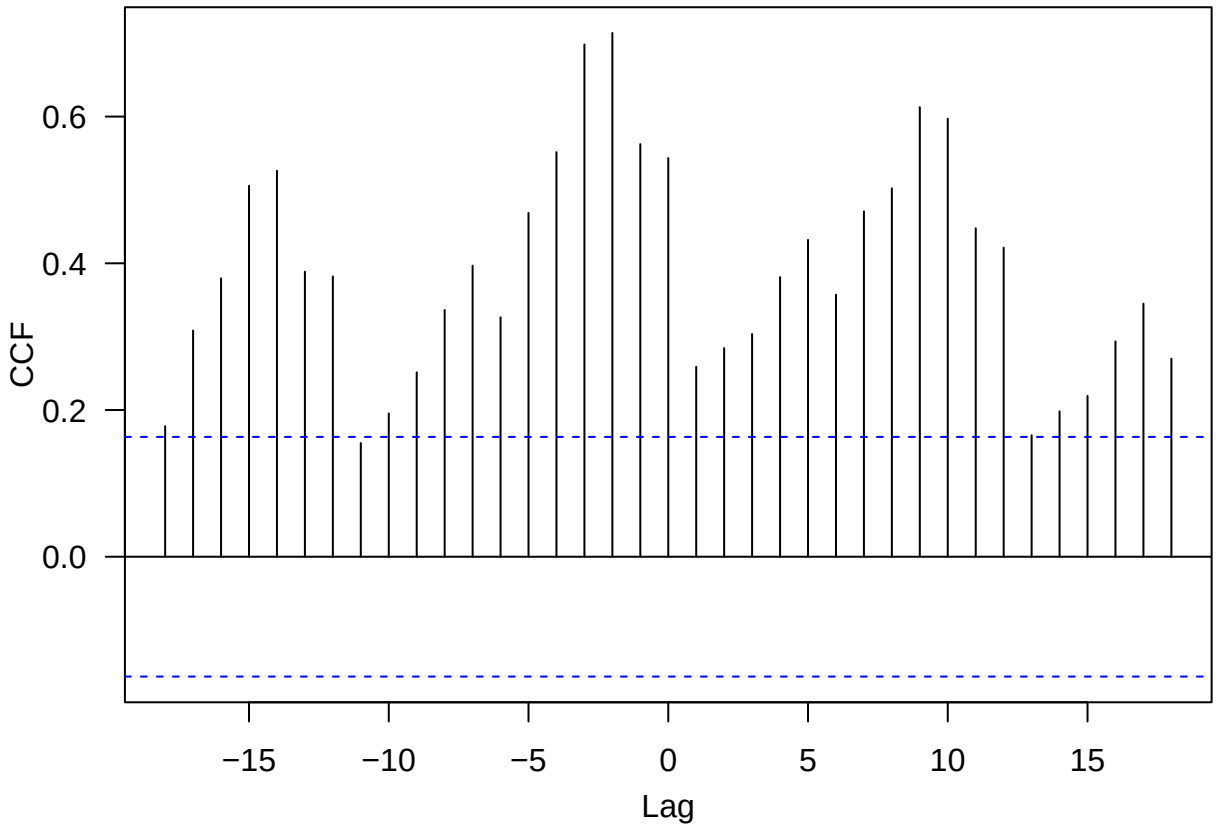
```
# Load and align the two time series
data(milk)
data(electricity)
milk.electricity <- ts.intersect(milk, log(electricity))

# Plot both series to examine trend and seasonality
par(mgp = c(2.2, 1, 0), mar = c(3.6, 3.6, 0.8, 0.6), family = "serif")
plot(milk.electricity, main = "")
```



We first examine the raw CCF to assess the apparent relationship between the two series.

```
par(las = 1, mgp = c(2.2, 1, 0), mar = c(3.6, 3.6, 0.8, 0.6))
ccf(as.vector(milk.electricity[, 1]),
as.vector(milk.electricity[, 2]), ylab = 'CCF', main = "")
```



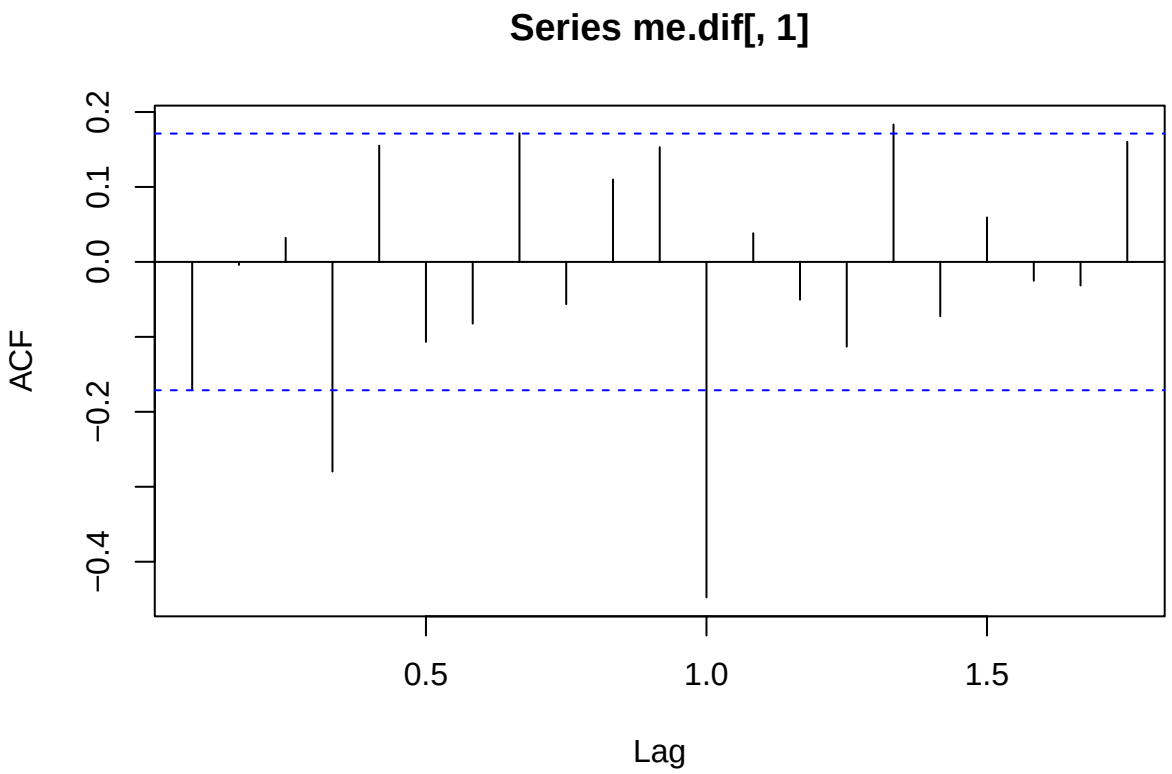
Interpretation: Strong cross-correlations may arise because both series share temporal patterns, such as trend or seasonality. These correlations do not necessarily indicate a direct relationship between milk production and electricity consumption.

Detrend and remove seasonality by differencing and applying prewhitening

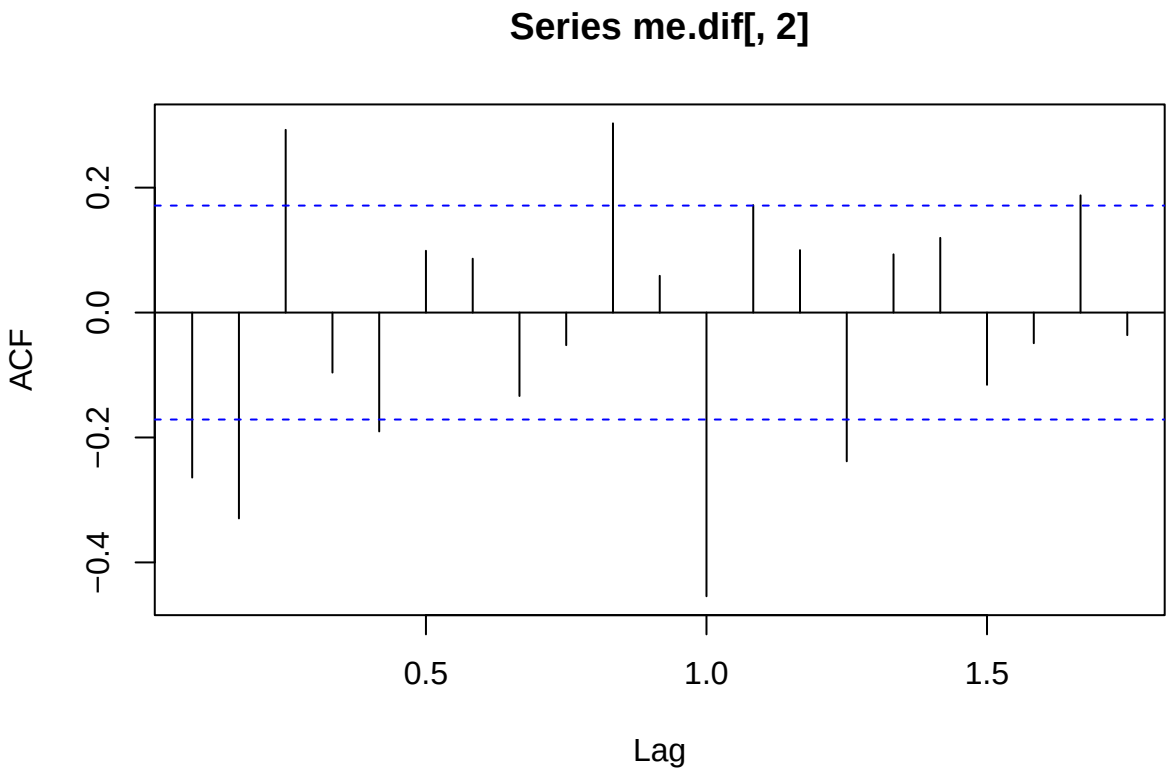
We first apply seasonal differencing (lag 12) and ordinary differencing (lag 1) to reduce nonstationarity. We then examine the ACFs and CCF before applying prewhitening to address remaining serial dependence.

```
# Apply seasonal and ordinary differencing to both series
me.dif <- ts.intersect(
  diff(diff(milk, 12)),
  diff(diff(log(electricity), 12))
)

# Examine autocorrelation after differencing
acf(me.dif[, 1])
```

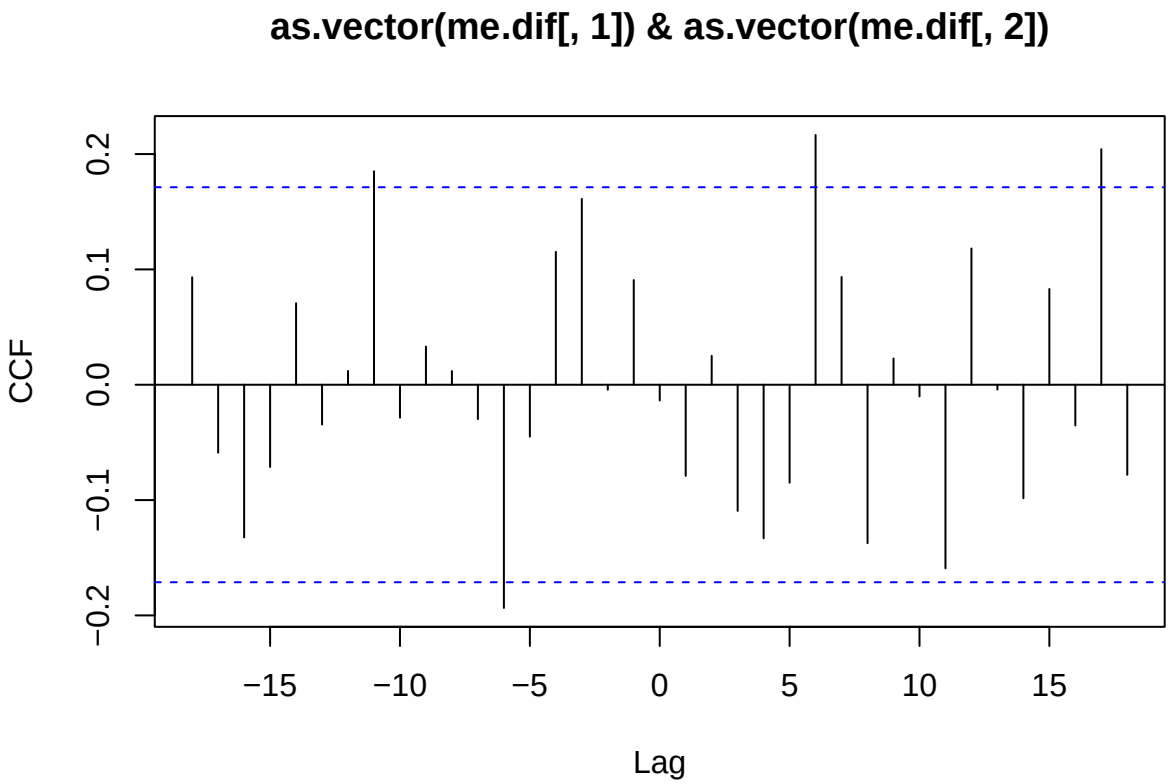



```
acf(me.dif[, 2])
```

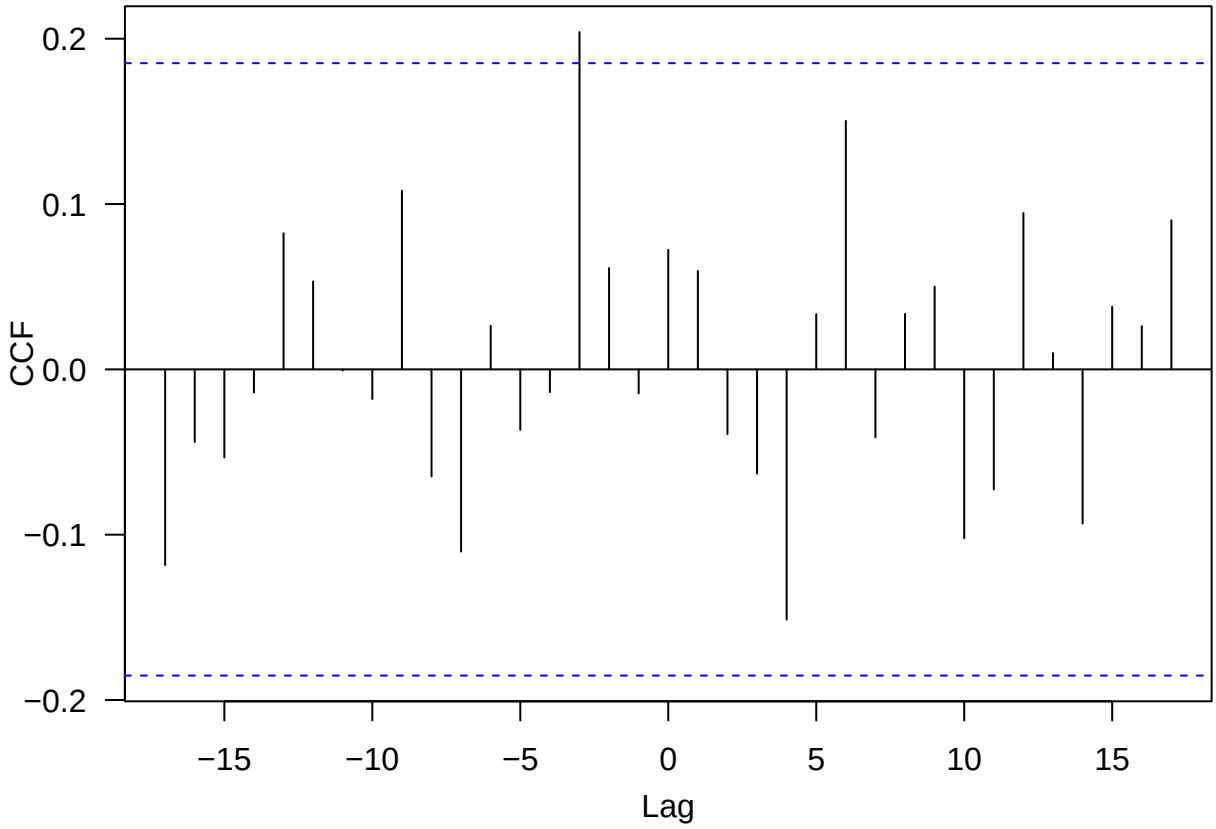


Next, we compare the CCF of the differenced series with the prewhitened CCF.

```
# Cross-correlation after differencing
ccf(as.vector(me.dif[, 1]),
    as.vector(me.dif[, 2]), ylab = "CCF")
```



```r
# Prewhiten to account for remaining serial dependence
par(las = 1, mgp = c(2.2, 1, 0), mar = c(3.6, 3.6, 0.8, 0.6))
prewhiten(as.vector(me.dif[, 1]),
          as.vector(me.dif[, 2]), ylab = "CCF")
```



Interpretation: Differencing removes trend and seasonal nonstationarity, while prewhitening addresses remaining autocorrelation. Comparing the CCFs before and after prewhitening helps distinguish apparent correlations driven by temporal dependence from potential cross-series relationships.

References

Jones, Philip D, and Michael E Mann. 2004. "Climate over Past Millennia." *Reviews of Geophysics* 42 (2).